

Automated Verification of Proofs in the Universal Composability Framework with Markov Decision Processes

Maxim Jourenko^{1*} and Marcus Völker²

¹ Institute of Science Tokyo, Japan
jourenko.m.606a@m.isct.ac.jp

² RWTH Aachen University, Germany
voelker@embedded.rwth-aachen.de

Abstract. Designing cryptographic protocols and proving these rigorously secure is an arduous and challenging task. Among the methods commonly used to prove security of cryptographic protocols, formalizing it in Canetti’s Universal Composability (UC) Framework offers several benefits: (1) Modular design, (2) demonstrating that security remains under arbitrary composition and concurrent execution, (3) the security against any computationally polynomially bound adversary. However, working within the UC Framework can be cumbersome, requires a long time commitment by the prover, and it is prone to errors. While utilization of proof assistants in Cryptography and IT Security is a prominent research area, proof assistants for UC are still in their infancy. Here we show our ongoing work to utilize model checking for verification of proofs in the UC Framework, which to the best of our knowledge is the first attempt to do so. In this work we (1) formally create a Markov Decision Process (MDP) encoding a given proof in the UC Framework, (2) define and prove notions of soundness and completeness for the constructed MDP, (3) implement a proof of concept and (4) demonstrate practical feasibility through experimental evaluation. In summary, in this work we lay out the formal foundations for model checking UC proofs and create a tool that can not only be used for proof verification but also as an assistant for developing proofs in the UC Framework.

1 Introduction

In the cryptographer’s toolkit there are two prominent frameworks for proving security of cryptographic systems and protocols. On the one side, there are game based proofs, where a game is defined based on a cryptographic system and a property it should fulfill. This game is played by an adversary and if they cannot win with more than negligible probability, the cryptographic system fulfills the desired property. An example of a game based security is *existential unforgeability*

* This work was supported by JST CREST Grant Number JPMJCR2113, Japan and Input Output Global.

against chosen message attacks (EU-CMA) for cryptographic signature schemes [20]. On the other side, there are simulation based proofs as Canetti’s Universal Composability Framework (UC) [14]. There, the prover creates a specification, called a functionality $\mathcal{F}$, of a cryptographic system they want to prove secure. They then attempt to show that a given protocol Π is indistinguishable to $\mathcal{F}$ from an outside perspective, called the Environment $\mathcal{Z}$ which is assisted by an adversary $\mathcal{A}$. An example of a functionality is $\mathcal{F}_{sig}$ and it has been shown equivalent to a EU-CMA secure signature scheme [9]. While in this case game based and simulation based notions for security of signature schemes have been shown to be equivalent, this is not the case in general. On the one side, UC provides more comprehensive guarantees to game based definitions: A system that was shown to implement a functionality is secure in any context, including if multiple instances of a protocol are executed in parallel or under composition of other protocols. Moreover, it is secure against attacks not known a-priori subject to the given security experiment defined by the prover. On the other side, a game based security definition requires the prover to limit themselves to showing security of a specific property, e.g. EU-CMA. However, the benefits of the UC framework come at a cost of a complex formalism where technical mistakes or nuances in the functionalities’ design are difficult to spot which is illustrated by the amount of iterations it took to achieve EU-CMA equivalence with a UC functionality [8, 1, 12, 13].

With the difficulties researchers face in providing rigorous security proofs, there is a growing body of work to attempt to mechanically, i.e. through a software program, prove security [2]. Of note is *EasyCrypt* [4] which is a tool to assist and verify game-based proofs utilizing Hoare logic [21] and variants thereof. However, the outlook regarding proof assistants for UC is rather bleak. Works that attempt to mechanize proofs in UC are EasyUC [14] as well as an approach to reason about adversarial complexity [3]. Both works are based on EasyCrypt. The drawback of these approaches is the significant effort to create a proof for a single cryptographic system that ranges in an order of magnitude of thousands of lines of code (LOC) [3] to tens of thousands [14], making these systems impractical and unscalable. In this work, we attempt to create a scalable proof assistant that allows formalization of cryptographic systems in the UC framework and mechanically verifies their correctness, utilizing model checking. We aim to lean more heavily on automated exploration of state spaces than the related work, which should offload some of what previously needed to be specified manually in a proof to an automated tool. As proof systems based on automated formal verification, such as model checking [15], usually have a natural way of diagnosing faulty inputs; model checking, for example, is capable of generating counterexamples that encode the precise reason why a system is incorrect. If we can leverage model checking for UC proof verification, we can then report counterexamples back to the user, which allows them to diagnose and fix the proof until it is correct.

1.1 Our Contributions

Our contributions are as follows:

- We formally show how to encode Turing Machines (TM) within a Markov Decision Process (MDP) and extend it to systems of Interactive Turing Machines (ITMs)
- We formally introduce how to encode a proof in the UC framework as a MDP M_{UC} and provide a property π_{UC} such that, if M_{UC} satisfies π_{UC} , then the UC proof is verified to be correct
- We formally define notions of *completeness* and *soundness* and prove that our proof system satisfies these
- We implement a proof-of-concept for our approach that consists of
 - A Domain Specific Language (DSL) that allows formalization of UC proofs and define protocols, functionalities and simulators akin to a pen-and-paper proof
 - A compiler that synthesizes a MDP written in the PRISM language [23] from a UC proof written in the DSL
- We showcase our approach by verifying the EasyUC variation [14] of the Diffie-Hellman based Key Exchange [16] and a Secure Message Communication protocol built on top of it as a use case
- We demonstrate practical feasibility through experimental evaluation of the above use case utilizing the PRISM model checker [23]

1.2 Related Work and Comparison

While there is a vast body of work for automated verification of game based proofs [2, 4], here we focus discussion of related work on approaches for mechanically verifying proofs made in the UC Framework. There are two approaches that both propose a proof assistant for UC proofs and are based on EasyCrypt[4].

EasyUC [14]. EasyUC is one of the first approaches in creating a proof assistant for UC. This work demonstrates formalization of UC in EasyCrypt and showcases it by proving secure a Diffie-Hellman key exchange as well as a secure message communication protocol based on it. However, while the authors succeed in their goal of formulizing UC, EasyUC does not have facilities to reason about adversarial complexity. While the version proposed is unscalable, as it required writing 18000 lines of code, a DSL has been implemented since³ reducing the required lines of code to less than about 600.

Adversarial Complexity Analysis [3]. Similar to EasyUC, Barbosa et al. utilize EasyCrypt to formalize a UC proof and use the same use case as EasyUC, i.e. secure message communication and Diffie-Hellman key exchange. However, focus of this work is reasoning about adversarial complexity, the missing component of EasyUC. In contrast to EasyUC, the authors use a less expressive variation of UC [11], which in-turn allows them to reduce the amount of effort by a prover by an order of magnitude to 2300 lines of code.

³ <https://github.com/easyuc/EasyUC>

SaUCy [24] SaUCy is a tool for automating UC proofs. It defines its own calculus, the Interactive Lambda Calculus (ILC), and similar to the work of Barbosa et al. is defined within a simpler variation of the UC framework [11]. As future work the authors propose fuzz testing as a potential means for automatic verification.

This work. In contrast to related works, we utilize model checking for proof verification instead of Hoare Logics. This has the benefit of automation of proof verification and, moreover, it additionally allows for creation of counterexamples which goes beyond simple verification and instead provides a valuable tool for creation of UC proofs themselves.

1.3 Outline of the Paper

In the following, we first provide the relevant background in Section 2. Then, we first formalize how to encode TMs and ITMs as MDPs in Section 3 before doing a step-by-step construction of an MDP that encodes a proof in the UC framework in Section 4. Afterwards we do both, theoretical analysis in Section 5 as well as experimental evaluation in Section 6. Finally, we conclude and discuss potential future work in Section 7.

2 Background

In this section we introduce the relevant background for this work. First, we show the UC framework in Section 2.1, the Decisional Diffie-Hellman Assumption in Section 2.2, followed by reviewing model checking and temporal logic in Section 2.3 and MDPs in Section 2.4.

2.1 The Universal Composability Framework [8]

For a proof in the UC Framework, a prover defines the following: (1) a protocol Π which is the cryptographic system to be analyzed, executed between $n \in \mathcal{N}$ parties, (2) a functionality $\mathcal{F}$ describing the ideal behavior of the cryptographic system in the form of a trusted third party that executes the cryptographic system on behalf of the parties and (3) a simulator $\mathcal{S}$ that takes a black-box adversary $\mathcal{A}$ as parameter and converts every attempted attack of the adversary onto the protocol into an attack onto the functionality within polynomial time. Each party can be identified using a unique party identifier pid . Formally, all these elements are Interactive Turing Machines (ITMs) whose computational resources are polynomially bounded. A Probabilistic Polynomial-Time (PPT) ITM called the *environment* $\mathcal{Z}$ receives access to a black box, either one containing a protocol instance of Π , the *real world*, or one containing the ideal protocol with $\mathcal{F}$, the *ideal world*. $\mathcal{Z}$ can interact with this black box, drive its execution, and receive its outputs. In this work, we refer to the outputs received by the environment as the *transcript*. At some point $\mathcal{Z}$ concludes its execution. Only

thereafter it reviews the output it received and outputs a value from $\{0, 1\}$ to indicate whether it interacted with the real or ideal world.

The environment is assisted by an adversary $\mathcal{A}$ who can corrupt parties, i.e. make them misbehave. The adversary represents the potential of arbitrary attacks executed against the protocol. The security proof succeeds if the environment can distinguish between real and ideal worlds with at most negligible probability, regardless of the behavior of the environment and adversary. This proof shows that any attack on the protocol can be converted into an attack onto the functionality within polynomial time; thus, the protocol is at least as secure as the functionality in the face of a polynomially bounded adversary, regardless of the context it is employed in.

The computational model for the entities within a UC proof are Interactive Turing Machines (ITMs). These are Turing Machines with four tapes: (1) an input tape on which it receives incoming messages, (2) an output tape, and (3) a backdoor tape used for communication with $\mathcal{A}$ and (4) a read-only and read-once tape containing random input. At the beginning, the environment is executed, who then can send messages to the world's interfaces and the adversary. When a message is sent, it is written on the input tape of the recipient. The sender of a message to an ITM is its caller. As soon as an ITM receives a message on its input tape it is executed and, in turn can send further messages. Once an ITM terminates, it optionally writes a message onto its output tape and then returns control to its caller.

UC Emulation. Let $\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}}$ be a protocol's Π real execution in presence of adversary $\mathcal{A}$ and $\text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$ an execution of functionality $\mathcal{F}$ in presence of simulator $\mathcal{S}$. Executions are controlled by environment $\mathcal{Z}$ given its input z , randomness r and a security parameter κ forming two *binary* ensembles

$$\begin{aligned}\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}} &= \{\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}}(\kappa, z, r)\}_{\kappa \in \mathbb{N}, z \in \{0, 1\}^*} \\ \text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}} &= \{\text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}(\kappa, z, r)\}_{\kappa \in \mathbb{N}, z \in \{0, 1\}^*}\end{aligned}$$

which are the outputs for $\mathcal{Z}$ from both executions respectively. It holds that Π realizes $\mathcal{F}$ if $\forall \text{PPT } \mathcal{Z}, \forall \text{PPT } \mathcal{A} \exists \text{PPT } \mathcal{S}$ such that for any $c \in \mathbb{N} \exists k, k_0 \in \mathbb{N}, \kappa > k_0$ such that $|\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}} - \text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}| < \kappa^{-c}$ written as $\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}} \approx_c \text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$. Informally, there exists no polynomial in κ such that the term $|\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}} - \text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}|$ is larger than the inverse of that polynomial.

Dummy Adversary. In this work we consider the UC framework with a *dummy* adversary, i.e. an adversary that forwards all messages and instructions from and to $\mathcal{Z}$ directly. It has been shown that demonstrating security in the presence of a dummy adversary is equivalent to demonstrating security in the presence of an arbitrary adversary in UC 2020 Claim 11 [10]. Note that utilizing a dummy adversary removes the quantifier for $\mathcal{A}$ in the UC emulation definitions.

2.2 Decisional Diffie-Hellman Assumption

Let $\mathcal{G}$ be a cyclic group of order $q \in \mathbb{N}$ and g be a generator of $\mathcal{G}$. Moreover, let $a, b, c \in \mathbb{N}$ be random numbers sampled from a (uniform) random distribution. Moreover, let $\mathcal{A}_{\text{DDH}}$ be a PPT algorithm taking a tuple of form (x, y, z) where $x, y, z \in \mathbb{N}$ as input and outputs $\{0, 1\}$. Then it holds that for $n, n_0 \in \mathbb{N}$ and security parameter κ $\exists n_0$ such that $\forall n > n_0$ holds that $\forall \mathcal{A}_{\text{DDH}} : |[\Pr(\mathcal{A}_{\text{DDH}}(g^a, g^b, g^{ab}) = 1)] - [\Pr(\mathcal{A}_{\text{DDH}}(g^a, g^b, g^c) = 1)]| < n^{-\kappa}$.

2.3 Model Checking and Temporal Logic

Model checking, in its most abstract form, is to determine whether the model $\mathcal{M}$ of a system satisfies a given specification φ , commonly written $\mathcal{M} \models \varphi$. Depending on the formalisms used to describe $\mathcal{M}$ and φ , different solvers using different algorithms can be applied; there is no general model checking algorithm that covers all kinds of specifications, and even for a given formalism, much research has gone into developing sophisticated algorithms to tackle the inherent complexity of model checking [15]. This complexity comes from the fact that, at least in the worst case, model checking requires investigating all possible state configurations of $\mathcal{M}$, which is combinatorially infeasible.

Although there are many different formalisms for specifications φ , we do not need the full expressive power of a given formalism for our application. Hence, we use the language of linear temporal logic (LTL) [26] only as a formal framework. LTL talks about the set of possible paths through the model, i.e. starting in a given state and then going to a successor state repeatedly. LTL has a grammar that allows for expressing many different properties (a subset of ω -regular languages) of these paths, but we only used two fairly simple kinds of properties for the verification in this paper: In each of the cases, take x to be a formula in propositional logic using atomic symbols from the system. A formula of form $G x$ means that x has to hold *globally*, that is, in each state of each path through the system. A formula of form $F x$ has to hold *finally*, that is, in at least one state of each path through the system. For a more complete discussion of the capabilities of LTL beyond what we are using, we refer to literature [26].

While the model traditionally is given in form of a state transition system such as a Kripke structure [22], the stochastic properties of cryptographic protocols necessitate a formalism that can capture these properties in a natural way, which is why we use a Markov Decision Process, which we use as input model for the PRISM model checker [23].

2.4 Markov Decision Process [27, 5]

A *Markov Decision Process* (MDP) can be thought of as a combination of a traditional state transition system and a stochastic process such as a Markov chain [17]. It is an attractive formalism for describing cryptographic protocols, as it can capture both decisions made by the parties and the probabilistic properties of cryptographic primitives (such as hash collisions). These form the basis of a

UC proof which at the end asks the question whether the two worlds can be distinguished with at most negligible probability.

Formally, an MDP is a probabilistic automaton (S, A, P) where S is a finite set of states, A is a finite set of actions, and $P : S \times A \times S \rightarrow [0, 1]$ is a probabilistic transition function. In a (non-probabilistic) finite automaton, A would usually be referred to as Σ , the alphabet. It serves a similar purpose in an MDP, describing the external events that cause the system to switch from one state to the next. For each pair of state $s \in S$ and action $a \in A$, there is a discrete probability distribution that describes the probability of the next state $s' \in S$, given by P . That is, $P(s, a, s')$ is the probability that in state s , action a leads to state s' . Consequently, $\forall s \in S, a \in A \sum_{s' \in S} P(s, a, s') = 1$.

Other concepts commonly associated with MDPs, such as rewards and policies, are not relevant to our contribution and have been omitted here.

3 Encoding Turing Machines as MDPs

Here we formally define how we can encode an ITM as an MDP for the purpose of model checking. In the following, we are storing data within an MDP's state. For instance, if state s contains a datum $x \in \{0, 1\}^*$ we refer it by the notation $s.x$.

3.1 Encoding Turing Machines

Recall that Turing Machines (TM) are tuples of form $T = (Q, \Gamma, b, \Sigma, q_0, F, \delta)$ where Q is a set of states, Γ is a set of symbols, $b \in \Gamma$ is the blank symbol, $\Sigma \subseteq \Gamma \setminus b$ is the set of input symbols q_0 is the initial state whereas $F \subseteq Q$ is a set of final states and $\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is a state transition function. Moreover, the Turing Machine contains a discrete tape consisting of cells and is initially populated with an input i , and a tape head h pointing to a cell on the tape, initially set to the first cell containing i . Lastly, a TM maintains its current state $q \in Q$. When executed, at each step, it computes $\delta(q, [h]) = (q', \gamma', D)$, and overwrites the tape cell at position a with γ , sets its current state to a' and shifts the head h in direction $D \in \{L, R\}$ where L indicates a shift by one cell to the left, and R by one cell to the opposite direction.

In the following we encode a TM T within an MDP $M_T = (S, A, P)$. First, we define the following auxiliary functions: (1) $\text{state}_T : \{0, 1\}^* \rightarrow Q \cup \{\perp\}$ be a function that takes a bitstring e_q as input and outputs $q \in Q$ if e_q is an encoding of q and $\perp$ otherwise. (2) $\text{tape}_T : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \Gamma \cup \{\perp\}$ takes two bitstrings which are an encoding of a tape e_t , and an encoding of a tape position e_h and outputs $\gamma \in \Gamma$ where the tape contains an encoding of symbol γ at position h if e_t and e_h are valid encodings of tape and head positions and $\perp$ otherwise. (3) $\text{move}_T : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{L, R, \perp\}$ takes two head positions e'_h, e''_h as input, outputs L if e''_h encodes the tape position to the left of the one e'_h encodes, R if e''_h encodes the tape position to the right analogously and $\perp$ otherwise.

For encoding a TM we proceed as follows: Within each $s \in S$ we store encodings of the current state in $s.q$, the head position in $s.h$ and the tape in $s.t$. Moreover, $A = \{a\}$ contains only one action. The execution of the TM is encoded within the state transitions P as follows: Let $s, s' \in S$ be two states. Then $P(s, a, s') = 1$ if and only if the following holds: (1) $\text{state}_T(s.q) \notin F$, (2) $\delta(\text{state}_T(s.q), \text{tape}_T(s.t, s.h)) = (\text{state}_T(s'.q), \gamma, D)$, (3) $\text{tape}_T(s'.t, s.h) = \gamma$ and (4) $\text{move}_T(s.h, s'.h) = D$. Otherwise $P(s, a, s') = 0$. Lastly, if $\forall s, s' \in S, s \neq s' : P(s, a, s') = 0$ we set $P(s, a, s) = 1$, denoting it as a *dead-end* state.

3.2 Encoding Interactive Turing Machines

ITMs [18, 28, 19, 10] are variations of TMs used for modelling distributed systems. In this work we consider the definition used in the 2020 version of UC [10]. We note that its authors show in Proposition 7 [10] that a system of ITMs can be simulated by a regular TM. Thus, we can encode a system of ITMs within a MDP by encoding a TM that simulates the system of ITMs respectively.

As we consider MDPs as finite for practical reasons, there is a contradiction above: Exact encoding of the tape of a TM requires an infinite amount of states. In the context of UC, though, the ITMs are bounded with a time predicate $T(\kappa)$ where κ is the security parameter. Therefore, we can bound our MDP M_T by discarding all states such that there exists no path over P with non-zero probability that is shorter than $k \cdot T(\kappa)$ for a chosen k . Formally, $M_{lim} = (S_{lim}, A, P_{lim})$

$$S_{lim} = \left\{ s \in S \mid \exists \pi \in S^* \left(\begin{array}{l} \pi[0] = s_0 \wedge \pi[|\pi| - 1] = s \\ \wedge |\pi| < k \cdot T(\kappa) \wedge \prod_{(s', a, s'') \in \pi} P(s', a, s'') > 0 \end{array} \right) \right\}$$

$$P_{lim} : S_{lim} \times A \times S_{lim} \rightarrow [0, 1], (s, a, s') \mapsto P(s, a, s')$$

As all of the states only contain finitely many successors in P with a non-zero probability (c.f. Section 3.1), finite reachability ensures that the MDP is finite as well.

4 Modelling of UC

In the following, we consider two *worlds* $\mathcal{I}$ and $\mathcal{R}$ as they would be specified for a UC proof, i.e. a system of ITMs including an environment $\mathcal{Z}$ steering its execution, and the MDPs $M_{\mathcal{I}} = (S_{\mathcal{I}}, A_{\mathcal{I}}, P_{\mathcal{I}})$ and $M_{\mathcal{R}} = (S_{\mathcal{R}}, A_{\mathcal{R}}, P_{\mathcal{R}})$ which encode both systems of ITMs by encoding the TM that simulates them as in Section 3.2. Our aim is to create an MDP M_{UC} representing a UC proof and to use it as input for model checking to verify whether any PPT environment $\mathcal{Z}$ can distinguish between $\mathcal{I}$ and $\mathcal{R}$.

Whereas in the previous section we showed how a system of ITMs can be modelled as an MDP, in this section we address the remaining challenges of

encoding a proof in the UC framework as an MDP: (1) How to model comparison between ideal and real worlds, (2) how to model execution of $\mathcal{I}$ and $\mathcal{R}$ driven by $\mathcal{Z}$, (3) how to quantify over all possible environments, (4) how to address randomness, (5) all while ensuring M_{UC} remains finite and (6) how to construct a property π_{UC} that can be used to prove that there does not exist any environment that can distinguish between $\mathcal{I}$ and $\mathcal{R}$ with more than negligible probability. We construct MDP M_{UC} by creating intermediary MDPs addressing the above challenges step-by-step and starting with $M_{\mathcal{I}}$ and $M_{\mathcal{R}}$.

4.1 Desired Properties

In the following we define two properties M_{UC} and φ_{UC} should fulfill such that model checking can make useful statements about a proof in the UC framework. For one, we require *completeness*, i.e. if an environment exists that can distinguish between two worlds with non-negligible probability, then M_{UC} must not satisfy φ_{UC} . On the flipside, we require *soundness*, i.e. if M_{UC} does not satisfy φ_{UC} there must exist a polynomial time environment that can distinguish between the two worlds simulated in M_{UC} . More formally we define both properties as follows. We discuss whether M_{UC} fulfills these properties in Section 5.

Definition 1 (Completeness). *Given a MDP M , property φ and systems of ITMs $\mathcal{I}$ and $\mathcal{R}$ defined according to the UC framework [7] where $\mathcal{I}$ contains functionality $\mathcal{F}$ and simulator $\mathcal{S}$ and where $\mathcal{R}$ contains protocol Π and dummy adversary $\mathcal{A}$, then (M, φ) satisfy completeness if $\exists \mathcal{Z}$ such that $\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}} \not\approx_c \text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$, then $M \not\models \varphi$.*

Definition 2 (Soundness). *Given a MDP M , property φ and systems of ITMs $\mathcal{I}$ and $\mathcal{R}$ defined according to the UC framework [7] where $\mathcal{I}$ contains functionality $\mathcal{F}$ and simulator $\mathcal{S}$ and where $\mathcal{R}$ contains protocol Π and dummy adversary $\mathcal{A}$, then (M, φ) satisfy Soundness if $M \not\models \varphi$ then $\exists \mathcal{Z}$ such that $\text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}} \not\approx_c \text{IDEAL}_{\mathcal{F}, \mathcal{S}, \mathcal{Z}}$.*

4.2 Comparing two Worlds

In order to be able to make any statements when comparing the executions of both $\mathcal{I}$ and $\mathcal{R}$ we must include both of them within M_{UC} . For this manner, we create an MDP $M_{\mathcal{I}, \mathcal{R}} = (S_{\mathcal{I}, \mathcal{R}}, A_{\mathcal{I}, \mathcal{R}}, P_{\mathcal{I}, \mathcal{R}})$ as a union of $M_{\mathcal{I}}$ and $M_{\mathcal{R}}$ as in the standard way of creating unions of automata where $S_{\mathcal{I}, \mathcal{R}} = S_{\mathcal{I}} \times S_{\mathcal{R}}$ such that $|S_{\mathcal{I}, \mathcal{R}}| = |S_{\mathcal{I}}| \cdot |S_{\mathcal{R}}|$ is finite. Note that this MDP is non-deterministic as both $\mathcal{I}$ and $\mathcal{R}$ can be executed in any order. This results in a large reachable statespace which makes model checking infeasible. Therefore, we make one adjustment to $M_{\mathcal{I}, \mathcal{R}}$ to reduce the amount of potential paths in $M_{\mathcal{I}, \mathcal{R}}$ to just one:

We observe that the executions of $\mathcal{I}$ and $\mathcal{R}$ are independent of each other and moreover, that an environment decides on and computes its output on which world it interacted with only after it executed $\mathcal{I}$ or $\mathcal{R}$ respectively. Thus, the order in which $\mathcal{I}$ and $\mathcal{R}$ are executed within $M_{\mathcal{I}, \mathcal{R}}$ does not impact the result of

executions of $\mathcal{I}$ and $\mathcal{R}$ and can be chosen arbitrarily. Now we proceed to adjust $P_{\mathcal{I},\mathcal{R}}$ as follows: First we define an ordering between both, $\mathcal{I}$ and $\mathcal{R}$. Without loss of generality let this ordering be $\mathcal{I} < \mathcal{R}$. Then, we remove all transitions where $P_{\mathcal{I},\mathcal{R}}$ proceeds to execute $\mathcal{R}$, if there exists any ITM, including the environment, in $\mathcal{I}$ which is active. Doing so, we remove the non-determinism in $M_{\mathcal{I},\mathcal{R}}$ by having $\mathcal{I}$ and $\mathcal{R}$ be executed sequentially. This ensures that there is at most one potential transition from each state.

4.3 A Fixed Environment $\mathcal{Z}$

As a preliminary step to reason across all possible environments, we extend $M_{\mathcal{I},\mathcal{R}}$ to $M_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ which encodes sequential execution of $\mathcal{I}$ and $\mathcal{R}$, but requires that both system of ITMs include the same environment $\mathcal{Z}$. Moreover, we adjust the transition function $P_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ of $M_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ to be as follows. Recall that $P_{\mathcal{I},\mathcal{R}}$ is chosen such that $\mathcal{I}$ and $\mathcal{R}$ are executed sequentially. We define $P_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ such that $\mathcal{Z}$'s execution is scheduled separately to the rest of the ITMs and only when no other ITM is active: (1) If any ITM in $\mathcal{I}$ with the exception of $\mathcal{Z}$ is active, then no ITM in $\mathcal{R}$ can proceed, (2) If any ITM in either $\mathcal{I}$ or $\mathcal{R}$ is active, then $\mathcal{Z}$ cannot proceed. Doing so, the scheduling in which ITMs are executed in $M_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ changes to first prioritize ITMs in $\mathcal{I}$, then $\mathcal{R}$ and lastly environments in both worlds $\mathcal{Z}$. (3) We remove all state transitions if both environments are active and only one environment would deactivate. Instead we add a state transition from any state in which the environments in both worlds would be deactivated if they proceed, to the state in which both ITMs proceed by one step. As a result, both environments simulated within the statespace of $M_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ issue instructions in tandem within one state transition. To construct $M_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ we only re-arranged transitions in comparison to $M_{\mathcal{I},\mathcal{R}}$ such that $M_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ remains finite.

4.4 Quantifying over all Environments

While it is possible to fix an ITM $\mathcal{Z}$ and verify whether it would be able to distinguish between $\mathcal{I}$ and $\mathcal{R}$ as in a UC proof, iterating over all possible environments is infeasible. We are forced to use abstraction of the environment to be able to quantify over all environments within an MDP without creating an infinitely large statespace or even just beyond what is feasible for model checking. Informally, we modify $M_{\mathcal{I},\mathcal{R}}$ to $M_{\mathcal{I},\mathcal{R},\mathcal{Z}}$ by (1) removing all information of the environments' states and tapes with exception of the input tapes that hold the outputs it received from the protocols, (2) replacing each instance in which both environments are activated with one state each representing the environment activation which (3) has one transition for each instruction the environment can issue to drive execution of $\mathcal{I}$ and $\mathcal{R}$ to a state where it issued that respective instruction in both $\mathcal{I}$ and $\mathcal{R}$. Doing so, we reduce each activation of the environments to one state and give the environments freedom of choice of their instructions through multiple non-deterministic state transitions. More formally we define an MDP $M_{\mathcal{I},\mathcal{R},\text{Env}}$ from $M_{\mathcal{I},\mathcal{R},\mathcal{Z}} = (S, A, P)$ as follows:

First, we set $A = \{a, a_0, a_1\} \cup A_{\text{Env}}$ where $A_{\text{Env}} = \{a_{\text{Env}} \mid a_{\text{Env}} = (m, \text{pid}) \text{ where } m \text{ is a message identifier and } \text{pid} \text{ is an identifier for an ITM in each } \mathcal{I} \text{ and } \mathcal{S}\}$, i.e. A contains a as defined in Section 3.1 as well as one action for each message and ITM the environment can send a message to. Note that the amount of ITMs in UC is polynomial in the security parameter [7], thus finite for any fixed security parameter. Moreover, note that a TM's alphabet is finite by definition. We ensure that the amount of messages is finite by omitting those messages where reading them would require more than a polynomial steps in the security parameter. Thus, A_{Env} is finite. We modify the statespace by removing both environments' state, tapes and head positions with exception of the tapes that hold the outputs they received from the protocols. Then, we remove all states in which either environment is executed which are those where both environments are active. Afterwards, for each state s which would have transitioned into a state where both environments are active, we create a state s_{Env} and adjust P such that there is a transition from s to s_{Env} at action a with probability 1. We further adjust P to create one transition from s_{Env} for each $a'_{\text{Env}} = (m, \text{pid}) \in A_{\text{Env}}$ into state s' on input a'_{Env} with probability 1. Here s' encodes the same state of the ITMs in s but has an additional message from the environment with message identifier m with the environment as sender on the input tape of the parties with identifier pid in both $M_{\mathcal{I}}$ and $M_{\mathcal{R}}$.

Lastly, we need to address parametrized messages. Without loss of generality, we assume that message parameters are encoded in a binary format, i.e. are in $\{0, 1\}^*$. Note that the parameter must consist of a finite amount of bits $c \in \mathbb{N}$ which is polynomial in the security parameter, since otherwise no computationally polynomially bounded ITM could read that parameter. Then, whenever an environment would choose a parameter of length c in state $s \in S$ we proceed as follows: We create a binary tree with s as root where the parameter is specified bit-by-bit. Then P has two transitions from s , one with action a_0 and probability 1 into state s_0 and one with action a_1 and probability 1 to s_1 representing fixing the first bit of the parameter to 0 or 1 respectively. Then let $j \in \{0, 1\}^{c'}$, $c \in \mathbb{N}$, $c' < c$. Then s_j is the state in which the parameter is specified to i with c' bits and P has two transitions from s_j with actions a_0 and a_1 respectively, i.e. to states s_{j0} and s_{j1} specifying the next bit respectively. Lastly, let $i \in \{0, 1\}^c$ be the chosen parameter. Then P has one transition from s_i to the state in which the environment would have sent its message with parameter i in state s . Note that this can be extended to multiple parameters by repeating this step for each parameter. This modification adds 2^c states to S which is finite. This approach can be generalized if parameters are encoded in a non-binary alphabet $\{m_0, m_1, \dots, m_n\}$ using n -ary trees.

However, this approach creates a few issues: (1) With this abstraction $M_{\mathcal{I}, \mathcal{R}, \text{Env}}$ includes paths for runs where the environment is computationally unbounded whereas the environment in the UC framework is computationally polynomially bounded and (2) while this modelling includes environments that can adapt their behavior to the outputs it received from driving the protocols, we only compare

executions for environments that act the same in both $\mathcal{I}$ and $\mathcal{R}$ even if their outputs differ. We address this in Section 4.6 and Section 5.

4.5 Randomness

Recall that each ITM has access to an input tape containing bits sampled from a uniform distribution. As a final step we derive $M_{UC} = (S, A, P)$ from $M_{\mathcal{I}, \mathcal{R}, \text{Env}}$ by modelling how ITMs read from these tapes. We assume that randomness is provided as a stream of bit values sampled from a uniform distribution of values in $\{0, 1\}$ and more complex distributions are computed by the ITMs using that bitstream as input. Then we modify $M_{\mathcal{I}, \mathcal{R}, \text{Env}}$ similar to how message parameter are picked by the environment but now utilizing transition probabilities instead of multiple actions:

We modify P such that, whenever in a state $s \in S$ an ITM would read a bit $i \in \{0, 1\}$ from a tape containing randomness and subsequently transitioning into states s_i respectively, we remove the original transitions that depend on i and replace them with transitions that go from state s into either s_i upon reading input a with probability 0.5 instead. Lastly, we remove the tape containing randomness from the MDP's state. While the ITMs we defined already have a means to utilize randomness, utilizing this approach has additional benefits: (1) We are able to remove the tape that contains random input from the statespace reducing the amount of potential states and making the MDP independent of the random input tape and (2) we can reason about probabilities during model checking. With the above modifications, the statespace remains finite, thus M_{UC} is finite.

4.6 The Indistinguishability Property

The *Indistinguishability Property* φ_{uc} is defined as an LTL formula such that if the property holds true on M_{UC} , then no computationally polynomial bounded environment exists which can distinguish between $\mathcal{I}$ and $\mathcal{R}$ with more than negligible probability. First, we create an intermediate formula φ_{int} of form Gx , i.e. x has to hold globally in each state of M_{UC} , where $x = y \vee z$. Then we set y and z such that y is true in a state $s \in S$ if s is not a dead-end state and z is *true*, if the environment's input tapes that contain the outputs of $\mathcal{I}$ and $\mathcal{R}$ are equal. Considering the abstraction of the environment used to construct M_{UC} , formula φ_{int} is *false* if there is any deviation between $\mathcal{I}$ and $\mathcal{R}$ on the same environment instructions, even when these are induced through different random inputs. This represents indistinguishability in the presence of a computationally unbounded environment, whereas the UC framework requires us to consider all ITMs to be computationally polynomial bounded.

However, φ_{int} is a rather strict property, modelling an environment that can distinguish $\mathcal{I}$ and $\mathcal{R}$ based on (1) two random values $r_{\mathcal{I}}, r_{\mathcal{R}} \in \mathbb{N}$ sampled from the same distribution or (2) based on two values $v_{\mathcal{I}}, v_{\mathcal{R}} \in \mathbb{N}$ even if doing so violates computational assumptions. To take the above cases in consideration, we construct a second intermediate formula $\varphi_{\text{int},2}$ which is of form $G(y \vee z')$

similar to φ_{int} . However, here z' requires additional inputs from a human prover who attempts to verify indistinguishability of $\mathcal{I}$ and $\mathcal{R}$ through model checking namely two additional lists L_R, L_I of inputs to create z' .

(1) L_R contains tuples of form $(r_{\mathcal{I}}, r_{\mathcal{R}})$ where $r_{\mathcal{I}}, r_{\mathcal{R}} \in \mathbb{N}$ are random values sampled from the same distribution in the ideal and real worlds respectively. To account for an environment $\mathcal{Z}$ not being able to distinguish these values, $\varphi_{\text{int},2}$ discards all paths where $r_{\mathcal{I}} \neq r_{\mathcal{R}}$ when evaluating distinguishability. (2) L_I contains tuples of form $(v_{\mathcal{I}}, v_{\mathcal{R}})$ where $v_{\mathcal{I}}, v_{\mathcal{R}} \in \mathbb{N}$ are values in the ideal and real worlds respectively and for which $\mathcal{Z}$ must break a computational assumption to be able to distinguish between them with more than negligible probability. To account for this, $\varphi_{\text{int},2}$ does not consider paths in which $v_{\mathcal{I}} \neq v_{\mathcal{R}}$ distinguishable.

At this point we are able to declare $\varphi_{\text{uc}} = \Pr_{\min}(\varphi_{\text{int},2}) \geq 1 - \text{negl}(\kappa)$, where $\text{negl}(\kappa)$ is a negligible term with respect to the security parameter κ and which now includes reasoning about probabilities as follows: $\Pr_{\min}(\varphi_{\text{int},2})$ is the probability of formula $\varphi_{\text{int},2}$ to be true after resolving all non-deterministic choices in the MDP to minimize its probability⁴. Thus, formula φ_{uc} is true if the probability that an environment can distinguish $\mathcal{I}$ and $\mathcal{R}$ is negligible, even if the environment acts optimally.

5 Theoretical Analysis

In the following we provide theoretical analysis of our construction. First, in Section 5.1 we prove that our construction satisfies properties Completeness as in Definition 1 and Soundness – with some limitations – as in Definition 2. Thereafter, we analyze complexity of the MDP in Section 5.2.

5.1 Theorems and Proofs

Theorem 1. $(M_{\text{UC}}, \varphi_{\text{UC}})$ *satisfies completeness.*

Proof. Let $M_{\text{UC}} = (S, A, P)$. Assume there exists an environment $\mathcal{Z}$ that can distinguish between $\mathcal{I}$ and $\mathcal{R}$ with more than negligible probability. First, assume that $\mathcal{Z}$ issues the same instructions when executing $\mathcal{I}$ and $\mathcal{R}$. Recall that sampling of uniform bitstrings is modeled in M_{UC} utilizing probabilistic transitions in P as described in Section 4.5. Thus, by construction, for any random bitstring r that can be sampled, a corresponding state that represents it being sampled is reachable in M_{UC} . The same holds for any instructions created by $\mathcal{Z}$ based on z . For each potential instruction by $\mathcal{Z}$ there exists a corresponding action in A such that the state that results in issuing that instruction is reachable in M_{UC} . Thus in summary, for any randomness r and input to the environment z there exists a path from M_{UC} 's initial state to a state corresponding to the executions of $\mathcal{I}$ and $\mathcal{R}$ on z , until either a dead-end state is reached, or the transcripts of both $\mathcal{I}$ and $\mathcal{R}$ contain distinguishable values. Thus, whenever $\mathcal{Z}$ can reach a state with distinguishable transcripts, a corresponding path exists

⁴ <https://www.prismmodelchecker.org/manual/PropertySpecification/ThePOperator>

in M_{UC} . The probability of $\mathcal{Z}$ to distinguish $\mathcal{I}$ and $\mathcal{R}$ will be at most as high as the probability of an environment to distinguish that acts optimally. As $\mathcal{Z}$ can distinguish with non-negligible probability, an optimally acting environment can distinguish with non-negligible probability. It follows that $M_{\text{UC}} \not\equiv \varphi_{\text{UC}}$ if $\mathcal{Z}$ issues the same instructions when executing $\mathcal{I}$ and $\mathcal{R}$.

Now we extend the proof to include environments that can issue different instructions when executing $\mathcal{I}$ and $\mathcal{R}$. Note that, since z and r are equal for both executing $\mathcal{I}$ and $\mathcal{R}$, $\mathcal{Z}$ can only issue different instructions only after the outputs of $\mathcal{I}$ and $\mathcal{R}$ differ. As before, assume $\mathcal{Z}$ can distinguish between $\mathcal{I}$ and $\mathcal{R}$. That implies $\mathcal{Z}$ can issue instructions to at least one $\mathcal{W} \in \{\mathcal{I}, \mathcal{R}\}$ that produces a transcript for $\mathcal{W}$ that is distinguishable with non-negligible probability. Thus, there exists a set of states $S' \subseteq S$ reachable with non-negligible probability where the transcript for $\mathcal{W}$ is distinguishable, whereas the transcript for the counterworld $\overline{\mathcal{W}} \in \{\mathcal{I}, \mathcal{R}\} \setminus \{\mathcal{W}\}$ might not match the transcript produced by executing $\mathcal{Z}$. However, as the transcript for $\mathcal{W}$ is distinguishable and such a transcript is reachable with non-negligible probability, it follows that $M_{\text{UC}} \not\equiv \varphi_{\text{UC}}$.

Theorem 2. *($M_{\text{UC}}, \varphi_{\text{UC}}$) satisfies soundness if $|A|$ is polynomial in security parameter κ and there exists a path in M_{UC} where a distinguishable state is reached after the environment issued at most $\mathcal{O}(1)$ instructions.*

Proof. Let $M_{\text{UC}} = (S, A, P)$ and assume that $M_{\text{UC}} \not\equiv \varphi_{\text{UC}}$. Then, there must exist a set of states S_{dist} reachable in M_{UC} from where the environment can distinguish. As ϕ_{UC} considers lists L_R and L_I it must hold that (1) $\mathcal{Z}$ does not distinguish between two random values $(r_0, r_1) \in L_R$ and does not distinguish based on two values $(v_0, v_1) \in L_I$ that would require breaking a computational assumption. Thus, $\mathcal{Z}$ can distinguish with non-negligible probability of at least p_a using a polynomial time algorithm $\mathcal{A}_{\text{dist}}$ in κ . Let p_s be the probability to reach any state S_{dist} from M_{UC} 's initial state. Then, the probability for M_{UC} to reach a distinguishable state is $p_s p_a$. Because $M_{\text{UC}} \not\equiv \varphi_{\text{UC}}$ it must hold that $\exists d \forall \kappa \in \mathbb{N} : p_s p_a > \kappa^{-c}$. Moreover, we assume that doing so, the environment issued $y \in \mathbb{N}, y \in \mathcal{O}(1)$ instructions in κ and $|A|$ is polynomial in κ , i.e. $\exists d \in \mathbb{N}$ such that $|A| < \kappa^d$. We can create an environment $\mathcal{Z}$ that can distinguish between $\mathcal{I}$ and $\mathcal{R}$ as follows:

When $\mathcal{Z}$ reaches a state it can distinguish by executing $\mathcal{A}_{\text{dist}}$, it does so. If $\mathcal{Z}$ is scheduled to execute by issuing an instruction, it picks a random instruction which corresponds to an action $a \in A$. As $\mathcal{A}_{\text{dist}}$ is a polynomial time algorithm and choosing a random value can be done in constant time, $\mathcal{Z}$ is a polynomial time ITM. The probability for $\mathcal{Z}$ to reach a state in which it can distinguish is $|\text{REAL}_{\Pi, A, \mathcal{Z}} - \text{IDEAL}_{\mathcal{F}, S, \mathcal{Z}}| \geq |A|^{-y} p_s p_a \geq (\kappa^d)^{-y} p_s p_a \geq \kappa^{-dy} \kappa^{-c} = \kappa^{-(c+dy)}$. Thus, for constant $c_s = c + dy \in \mathbb{N}$ it holds that for all $\kappa : |\text{REAL}_{\Pi, A, \mathcal{Z}} - \text{IDEAL}_{\mathcal{F}, S, \mathcal{Z}}| \geq \kappa^{-c_s}$. It follows that $\mathcal{Z}$ can distinguish between $\mathcal{I}$ and $\mathcal{R}$, i.e. $\text{REAL}_{\Pi, A, \mathcal{Z}} \not\approx_c \text{IDEAL}_{\mathcal{F}, S, \mathcal{Z}}$. It follows $(M_{\text{UC}}, \varphi_{\text{UC}})$ satisfies soundness.

When showing Theorem 2 we assume that the potential messages from an environment are polynomial in κ and the amount of rounds in which the environment issues messages is constant. We admit that this round limitation does

impact in which cases (M_{UC}, φ_{UC}) satisfies soundness, however, there are multiple avenues that we leave for future work to mitigate this issue. For one, instead of a naive construction for $\mathcal{Z}$ we could consider more elaborate methods used in MDP policy planning [6]. For another, M_{UC} cannot model protocols that go on for an arbitrary number of rounds as this would potentially create a non-finite state space. However, further state abstraction techniques could be employed to allow for arbitrary round protocols and environments.

5.2 Scalability

Model checking is a computationally expensive task as a model’s statespace most often is exponential in size. As we are not yet implementing state space abstraction with exception of the environment, there are multiple parameters which increase M_{UC} ’s state space exponentially. For one, random numbers or secret keys are often sampled in length of the security parameter κ which increases the state space exponentially in κ . Moreover, while the statespace increases linearly in the amount of choices $\mathcal{Z}$ has, i.e. linearly in the amount of actions $|A|$, the amount of actions themselves can be exponential in size, e.g. if the length of a message parameter has size of length κ . In addition, the state space increases exponentially in the amount of rounds in which the environment can send messages. Lastly, the state space increases depending on the protocol code, more specifically, the amount of branches within the code, e.g. if-statements. This has consequences for which proofs are feasible to verify: (1) Protocols should be constant round, (2) the choice of messages sent by $\mathcal{Z}$ should be polynomial, (3) code execution should branch at most a constant amount of times, (4) proofs can only be verified for low values for the security parameter κ explicitly. As a consequence, for proofs that require high values of κ to be secure, e.g. if security is asymptotic in κ , and where the message size of the environment is linear in κ , the proof would have to be executed for low values of κ and message sizes of $\mathcal{Z}$ and then it would require additional effort by a human prover to show these results can be extended to higher values of κ and messages sizes respectively. However, we argue that this issue is not solely the case for model checking but also inherent for code verification as proof assistants created in previous works [14, 3] require use of satisfiability solvers which tasks are NP hard in general. Nevertheless, these issues are not entirely prohibitive and we list potential solutions to this in Section 7.1. Even though model checking is challenging, we demonstrate viability of our approach on a realistical use case in Section 6.

6 Experimental Evaluation

We implemented a *proof of concept* to demonstrate the practical feasibility of the approach presented in this work. The proof of concept consists of (1) A grammar specifying a Domain Specific Language (DSL) written with ANTLR [25] in which a prover can write a UC proof with notation close to a pen and paper proof, (2) a compiler for UC proofs written in the DSL that synthesizes

a corresponding MDP, written in the PRISM language [23]. We showcase our tool by implementing the usecase described in Section 6.1, i.e. a Diffie-Hellman Key exchange and a Secure Message Communication protocols which required writing about 700 lines of code in the DSL. We then use our tool to create an MDP and thereafter verify it using the PRISM modelchecker [23]. Moreover, we lay out limitations of the current implementation in Section 6.2 before we present and discuss our results in Section 6.3.

Source code of the proof-of-concept and the implementation of the use case in the DSL are provided in the supplementary materials.

6.1 Use Case

As usecase to demonstrate the feasibility of our proof-of-concept implementation we implement two proofs: (1) A Diffie-Hellman Key Exchange and (2) Secure Message Communication (SMC). Both UC proofs have been, to the best of our knowledge, the benchmark for demonstrating all previous proof assistants for UC [14, 3]. We implement both proofs using variations used for Easy UC [14] and we instantiate the SMC protocol using the ElGamal cryptosystem. In either case we chose the length of the keys that are created to be equal the security parameter κ and do experimental evaluation for different values of κ in Section 6.3. The length of the message to be communicated in the SMC protocol, which is chosen by the environment, is chosen to be 1 bit. Security of both protocols relies on the DDH Assumption as laid out in Section 2.2. We argue that demonstrating security of the protocol under low values for κ by utilizing the DDH assumption extends directly to prove security of the protocols for high values of κ . Moreover, we argue that the proof is independent of the specific message the environment asks to transmit in the SMC protocol as its value affects the protocol transcript only by affecting indistinguishable values, i.e. the ciphertext transmitted between both parties, and it is then output at the end by the message’s recipient after decryption. The usecase is described in detail in Appendix A provided in the supplementary materials.

6.2 Limitations of the Implementation

The aim of our implementation is to provide a proof of concept. While we implement the approach described in this work, the current implementation does not yet support verification of *any arbitrary* UC proof, but instead is made to be sufficient for verifying our use case. Nevertheless, implementation of these aspects and removal of these limitations remains subject of future work. Moreover, while Section 4 defines M_{UC} closely to the definition of UC 2020 [10] using ITMs, the computational model simulated in the MDP created with our tool are not ITMs but instead something akin to a register machine for simplicity and ease of use. Note that the computational model of UC is not strictly limited to ITMs but any reasonable model that can account for computational complexity [10]. In the following, we lay out the limitations of the current proof-of-concept:

For one, the current implementation utilizes a simplified communication model and, moreover, does not yet make use of a call stack, limiting communication as follows: (1) ITMs do not give outputs to sub-routines via an output tape, but communicate any values through explicit messages, (2) the backdoor tape is stored implicitly by reusing the input tape and distinguishing by the messages' sender, (3) ITMs are limited to sending messages at the end of executing any code routine and limited to only one message. Moreover, we currently have no direct means to reason about adversarial complexity as in [3].

6.3 Experimental Evaluation

We evaluated our implementation on the two use cases (the Diffie-Hellman Key exchange and SMC) by generating PRISM instances with varying security parameters. We set a timeout of 10 minutes and increased the security parameter until PRISM would not find a result in the required time.

The evaluation graphs are split into two per use case: One investigating the size of the MDP and one investigating the run time of PRISM on the MDP. The results can be seen in Figures 1a, 1b, 2a and 2b

We can clearly see the exponential relationship between the security parameter and the size of the MDP. The key exchange uses three random numbers the length of the security parameter, while SMC uses two random numbers the length of the security parameter. Consequently, we expect a state space the size of $O(2^{3x}) = O(8^x)$ for the key exchange and $O(2^{2x}) = O(4^x)$ for SMC, which the experimental findings in Figures 1a and 2a support. Because of the slower scaling of the SMC instance, we were able to evaluate until a security parameter of $\kappa = 6$ compared to the security parameter $\kappa = 4$ for the key exchange.

The model checking scales linearly with model size, so we see the same exponential relationships for the run times in Figures 1b and 2b.

7 Conclusion and Future Work

In this work we introduced model checking for verification of proofs in the UC Framework. We showed how to perform model checking formally by creating a MDP that models the UC proof step-by-step, first modelling TMs and ITMs as MDPs and subsequently a full UC proof. We proved that our approach satisfies notions of completeness and soundness. Lastly, we demonstrated practical feasibility of our approach by implementing a proof of concept and verifying UC proofs for two protocols, a Diffie-Hellman key exchange as well as a secure message communication protocol. We stress that this is the first attempt of model checking UC proofs and it goes beyond proof verification but also enables counterexample generation as an aid for writing proofs in the UC framework which proved to be invaluable when writing the UC proofs for the use cases covered in this work.

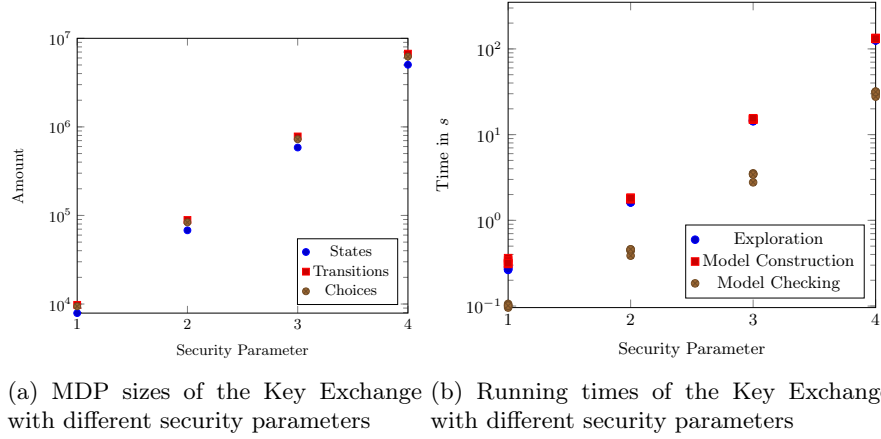


Fig. 1: Experimental evaluation of the Diffie-Hellman Key Exchange use case.

7.1 Future Work

Even though model checking is challenging as laid out in our discussion of scalability in Section 5.2, we demonstrated the feasibility of our approach in Section 6.3 by model checking the use case laid out in Section 6.1. However, further challenges remain and venues for future work are plentiful. For one, we can make further improvements to the proof-of-concept implementation by (1) improving the expressiveness of the DSL and (2) improving on the modelling of communication to remove limitations as laid out in Section 6.2. Furthermore, we currently can only demonstrate *soundness* for our approach with limitations on the environment. Exploring methods of policy planning in MDPs [6] could alleviate these restrictions. Another avenue of future work is to attempt to reduce the potential state space of the created MDP in an attempt to remove sources of exponential growth of the state space. For one, we can consider further venues of state space abstraction: (1) Usually, if not almost always, for a UC proof the exact values of random numbers are irrelevant, thus abstracting away from specific values for random numbers while modelling resulting probabilities correctly might help make the resulting MDP's state space independent of security parameter κ . This approach could also be used for some message parameters of the environment to reduce the state space depending on the amount of choices the environment has. (2) Employ a higher level abstraction to the state space such that, any round of instructions from the environment does not result in a state transition into a state outside of this abstract state space. This would allow for model checking of protocols with arbitrary rounds and make size of the MDP independent of the rounds of messages sent by the environment. In addition this is a possible approach to satisfy *soundness* without restrictions on the environment. (3) Lastly, exploring utilization of symbolic execution can further reduce the state space of the MDP. Finally, while we currently translate our DSL into the input language

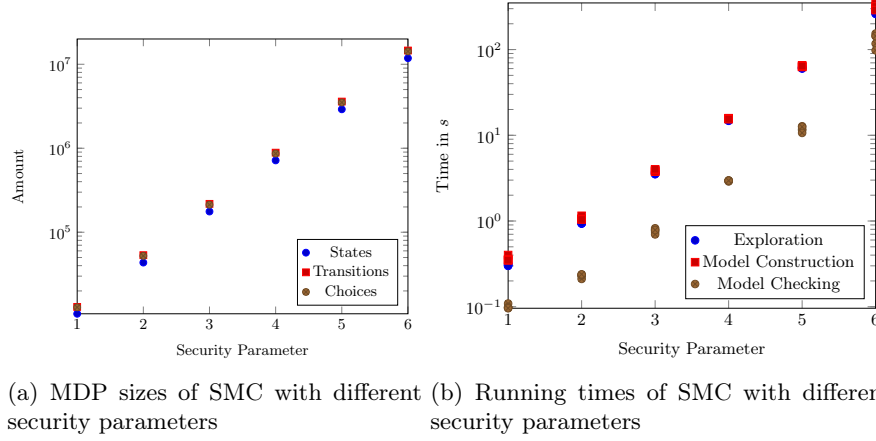


Fig. 2: Experimental evaluation of the Secure Message Communication use case.

of PRISM, we inspect PRISM’s output manually to check whether the proof is correct and if not, we also manually inspect the counterexample. In the interest of usability, we would like to improve our tooling so the output is automatically processed and the counterexample is converted into a representation that makes it easier to use when fixing incorrect proofs.

References

1. Michael Backes and Dennis Hofheinz. How to break and repair a universally composable signature functionality. In *International Conference on Information Security*, pages 61–72. Springer, 2004.
2. Manuel Barbosa, Gilles Barthe, Karthik Bhargavan, Bruno Blanchet, Cas Cremers, Kevin Liao, and Bryan Parno. Sok: Computer-aided cryptography. In *2021 IEEE symposium on security and privacy (SP)*, pages 777–795. IEEE, 2021.
3. Manuel Barbosa, Gilles Barthe, Benjamin Grégoire, Adrien Koutsos, and Pierre-Yves Strub. Mechanized proofs of adversarial complexity and application to universal composability. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 2541–2563, 2021.
4. Gilles Barthe, Benjamin Grégoire, Sylvain Heraud, and Santiago Zanella Béguelin. Computer-aided security proofs for the working cryptographer. In *Annual Cryptology Conference*, pages 71–90. Springer, 2011.
5. Richard Bellman. Dynamic programming and stochastic control processes. *Information and Control*, 1(3):228–239, 1958.
6. Craig Boutilier, Thomas Dean, and Steve Hanks. Decision-theoretic planning: Structural assumptions and computational leverage. *Journal of Artificial Intelligence Research*, 11:1–94, 1999.
7. R. Canetti. Universally composable security: a new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pages 136–145, 2001.

8. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pages 136–145. IEEE, 2001.
9. Ran Canetti. Universally composable signature, certification, and authentication. In *Proceedings. 17th IEEE Computer Security Foundations Workshop, 2004.*, pages 219–233. IEEE, 2004.
10. Ran Canetti. Universally composable security. *Journal of the ACM (JACM)*, 67(5):1–94, 2020.
11. Ran Canetti, Asaf Cohen, and Yehuda Lindell. A simpler variant of universally composable security for standard multiparty computation. In *Advances in Cryptology—CRYPTO 2015: 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16–20, 2015, Proceedings, Part II 35*, pages 3–22. Springer, 2015.
12. Ran Canetti and Hugo Krawczyk. Universally composable notions of key exchange and secure channels. In *Advances in Cryptology—EUROCRYPT 2002: International Conference on the Theory and Applications of Cryptographic Techniques Amsterdam, The Netherlands, April 28–May 2, 2002 Proceedings 21*, pages 337–351. Springer, 2002.
13. Ran Canetti and Tal Rabin. Universal composition with joint state. In *Advances in Cryptology—CRYPTO 2003: 23rd Annual International Cryptology Conference, Santa Barbara, California, USA, August 17–21, 2003. Proceedings 23*, pages 265–281. Springer, 2003.
14. Ran Canetti, Alley Stoughton, and Mayank Varia. Easyuc: Using easycrypt to mechanize proofs of universally composable security. In *2019 IEEE 32nd Computer Security Foundations Symposium (CSF)*, pages 167–16716. IEEE, 2019.
15. Edmund M. Clarke and E. Allen Emerson. Design and synthesis of synchronization skeletons using branching time temporal logic. In Dexter Kozen, editor, *Logics of Programs*, pages 52–71, Berlin, Heidelberg, 1982. Springer Berlin Heidelberg.
16. Whitfield Diffie and Martin E Hellman. New directions in cryptography. In *Democratizing Cryptography: The Work of Whitfield Diffie and Martin Hellman*, pages 365–390. 2022.
17. Arnoldo Frigessi and Bernd Heidergott. *Markov Chains*, pages 772–775. Springer Berlin Heidelberg, Berlin, Heidelberg, 2011.
18. Oded Goldreich. Foundations of cryptography: Basic tools, vol. 1, 2001.
19. Shafi Goldwasser, Silvio Micali, and Chales Rackoff. The knowledge complexity of interactive proof-systems. In *Providing sound foundations for cryptography: On the work of shafi goldwasser and silvio micali*, pages 203–225. 2019.
20. Shafi Goldwasser, Silvio Micali, and Ronald L Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on computing*, 17(2):281–308, 1988.
21. Charles Antony Richard Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
22. Saul Kripke. Semantical considerations on modal logic. *Acta Philosophica Fennica*, 16:83–94, 1963.
23. M. Kwiatkowska, G. Norman, and D. Parker. PRISM 4.0: Verification of probabilistic real-time systems. In G. Gopalakrishnan and S. Qadeer, editors, *Proc. 23rd International Conference on Computer Aided Verification (CAV’11)*, volume 6806 of *LNCS*, pages 585–591. Springer, 2011.
24. Kevin Liao, Matthew A Hammer, and Andrew Miller. Ilc: a calculus for composable, computational cryptography. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 640–654, 2019.

25. Terence Parr. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, 2nd edition, 2013.
26. Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science (sfcs 1977)*, pages 46–57, 1977.
27. S. Schneider and D. H. Wagner. Error detection in redundant systems. In *Papers Presented at the February 26-28, 1957, Western Joint Computer Conference: Techniques for Reliability*, IRE-AIEE-ACM '57 (Western), page 115–121, New York, NY, USA, 1957. Association for Computing Machinery.
28. Douglas Wikström. Simplified universal composability framework. In *Theory of Cryptography Conference*, pages 566–595. Springer, 2015.

A The Use Case

In the following, we denote $x \leftarrow_{\$} \{0, 1\}^\kappa$ as assigning a random value to variable x , sampled from a uniform distribution from the set of bitstrings in $\{0, 1\}^\kappa$ where κ is the security parameter.

A.1 Diffie-Hellman Key Exchange

Functionality $\mathcal{F}_{\text{KE}}$ is an ideal specification of the Diffie-Hellman key exchange and functionality $\mathcal{F}_{\text{FWD}}$ models message forwarding subject to arbitrary delay imposed by $\mathcal{A}$. Both functionalities are variants as used in EasyUC. The real key exchange protocol is shown in Figure 5 and 6 which display the protocols of the key exchange initiator $\mathcal{P}_0$ and their counterpart $\mathcal{P}_1$. Both parties communicate through two instances of $\mathcal{F}_{\text{FWD}}$: Instance $\mathcal{F}_{\text{FWD},0}$ forwards a message from $\mathcal{P}_0$ to $\mathcal{P}_1$ whereas $\mathcal{F}_{\text{FWD},1}$ forwards a message into the other direction respectively. Message delivery is controlled by $\mathcal{A}$ through the backdoor tape of $\mathcal{F}_{\text{FWD}}$. $\mathcal{P}_0$ and $\mathcal{P}_1$ generate exponents $a \in \mathbb{N}$ and $b \in \mathbb{N}$ respectively and output $g_c = g^{ab} \bmod p$ as the negotiated key, where g is a generator and primitive root modulo p . Functionality $\mathcal{F}_{\text{KE}}$ for the ideal key exchange is shown in Figure 3 $\mathcal{F}_{\text{KE}}$ has first awaits a message from the protocol initiator $\mathcal{P}_0$ and notifies $\mathcal{S}_{\text{KE}}$ as shown in Figure 7 who simulates $\mathcal{P}_0$ initiating the protocol according to the real protocol as in Figure 5. Of note is that while $\mathcal{S}_{\text{KE}}$ simulates $\mathcal{P}_0$ and $\mathcal{P}_1$, thus can output the same exponents a and b , functionality $\mathcal{F}_{\text{KE}}$ outputs a key that is unrelated to the key exchange. However, utilizing the decisional Diffie-Hellman assumption as in Section 2.2, the environment is not able to distinguish between both keys.

A.2 Secure Message Communication

The secure message communication functionality $\mathcal{F}_{\text{SMC}}$ is described in Figure 8, whereas the SMC protocol executed by $\mathcal{P}_1$ and $\mathcal{P}_2$ are shown in figures 9 and 10 respectively. Both, the functionality and protocol are variations as used in EasyUC [14]. Functionality $\mathcal{F}_{\text{SMC}}$ first receives a message ($\text{SEND}, \text{sid}, \mathcal{P}', \text{msg}$) which instructs sending of message msg to another party $\mathcal{P}'$. Upon receipt of such a message $\mathcal{F}_{\text{SMC}}$ notifies $\mathcal{S}_{\text{SMC}}$ through sending a message SMC including both parties' identities, but without leaking message msg . Afterwards, upon receiving a DELIVER instruction from $\mathcal{S}_{\text{SMC}}$ it forwards the message to its recipient $\mathcal{P}'$. The protocol executed between both parties that realizes $\mathcal{F}_{\text{SMC}}$ requires them to first, utilize $\mathcal{F}_{\text{KE}}$ to agree on a common secret key k which is then used as a one-time pad by $\mathcal{P}_1$ to encrypt message msg . The resulting ciphertext is then sent to $\mathcal{P}_2$ utilizing the forwarding functionality $\mathcal{F}_{\text{FWD}}$. Upon receipt of it, $\mathcal{P}_2$ can retrieve the original message utilizing the previously created key k . The simulator $\mathcal{S}_{\text{SMC}}$ for $\mathcal{F}_{\text{SMC}}$ is detailed in Figure 11. It's role is to simulate the SMC protocol executed by $\mathcal{P}_1$ and $\mathcal{P}_2$ as well as the functionalities $\mathcal{F}_{\text{KE}}$ and $\mathcal{F}_{\text{FWD}}$ involved in the protocol and interact with $\mathcal{A}$ corresponding to it. For definition of the protocols in Figure 9 and 10 we assume two algorithms (E, D) which are (1) an encryption algorithm $\text{E} : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ that takes a

Key exchange functionality $\mathcal{F}_{\text{KE}}$

Variables and Initialization:

- State $s = 0$
- Exponent $d = 0$ and Key $g_d = 0$

Phase 1: Upon receiving input $(\text{KEY}, \text{sid}, \mathcal{P}_1)$ from $\mathcal{P}_0$:

- 1: **if** $s = 0$ **then**
- 2: $s \leftarrow 1$
- 3: $d \leftarrow_{\$} \{0, 1\}^\kappa$
- 4: $g_d \leftarrow g^d \bmod p$
- 5: Send $(\text{KEY0}, \text{sid}, \mathcal{P}_0, \mathcal{P}_1)$ to $\mathcal{S}_{\text{KE}}$

Phase 1 Delivery: Upon receiving input $(\text{KEY0}, \text{sid})$ from $\mathcal{S}_{\text{KE}}$:

- 1: **if** $s = 1$ **then**
- 2: $s \leftarrow 2$
- 3: Send $(\text{Key}, \text{sid}, g_d)$ to $\mathcal{P}_1$

Phase 2: Upon receiving input (KEY, sid) from $\mathcal{P}_2$:

- 1: **if** $s = 2$ **then**
- 2: $s \leftarrow 3$
- 3: Send $(\text{KEY1}, \text{sid}, \mathcal{P}_0, \mathcal{P}_1)$ to $\mathcal{S}_{\text{KE}}$

Phase 2 Delivery: Upon receiving input $(\text{KEY1}, \text{sid})$ from $\mathcal{S}_{\text{KE}}$:

- 1: **if** $s = 3$ **then**
- 2: $s \leftarrow 4$
- 3: Send $(\text{Key}, \text{sid}, g_d)$ to $\mathcal{P}_0$

Fig. 3: The Diffie-Hellman Key Exchange functionality models the protocol execution of the key exchange protocol and, selects an arbitrary exponent d to compute a key g^d that is delivered to $\mathcal{Z}$.

Message forwarding functionality $\mathcal{F}_{\text{FWD}}$

Variables and Initialization:

- State $s = 0$
- Data $d = \perp$
- Sender $p_0 = \perp$ and recipient $p_1 = \perp$

Receipt: Upon receiving input $(\text{FWD}, \text{sid}, d', \mathcal{P}')$ from $\mathcal{P}$:

- 1: **if** $s = 0$ **then**
- 2: $s \leftarrow 1$
- 3: $p_0 \leftarrow \mathcal{P}, p_1 \leftarrow \mathcal{P}', m \leftarrow m', d \leftarrow d'$
- 4: Send $(\text{FWD}, \text{sid}, p_0, p_1, d)$ to $\mathcal{A}$

Delivery: Upon receiving input (FWD, sid) from $\mathcal{A}$:

- 1: **if** $s = 1$ **then**
- 2: $s \leftarrow 2$
- 3: Send $(\text{FWD}, \text{sid}, p_0, \text{data})$ to p_1

Fig. 4: The message forwarding functionality forwards a message from one party to another, subject delay imposed by $\mathcal{A}$.

KE Protocol for $\mathcal{P}_0$

Variables and Initialization:

- State $s = 0$
- Exponent $a = \perp$

Key Request: Upon receiving input $(\text{KEY}, \text{sid}, \mathcal{P}_1)$ from $\mathcal{Z}$:

- 1: **if** $s = 0$ **then**
- 2: $s \leftarrow 1$
- 3: $a \leftarrow_{\$} \{0, 1\}^\kappa$
- 4: Send $(\text{FWD}, \text{sid}, \mathcal{P}_1, g^a \bmod p)$ to $\mathcal{F}_{\text{FWD},0}$

Key Delivery: Upon receiving input $(\text{FWD}, \text{sid}, \mathcal{P}_1, k_1)$ from $\mathcal{F}_{\text{FWD},1}$:

- 1: **if** $s = 1$ **then**
- 2: $s \leftarrow 2$
- 3: Send $(\text{FWD}, \text{sid}, \text{KEY}, k_1^a \bmod p)$ to $\mathcal{Z}$

Fig. 5: The protocol for the initiator of the Diffie-Hellman key exchange in the real world.

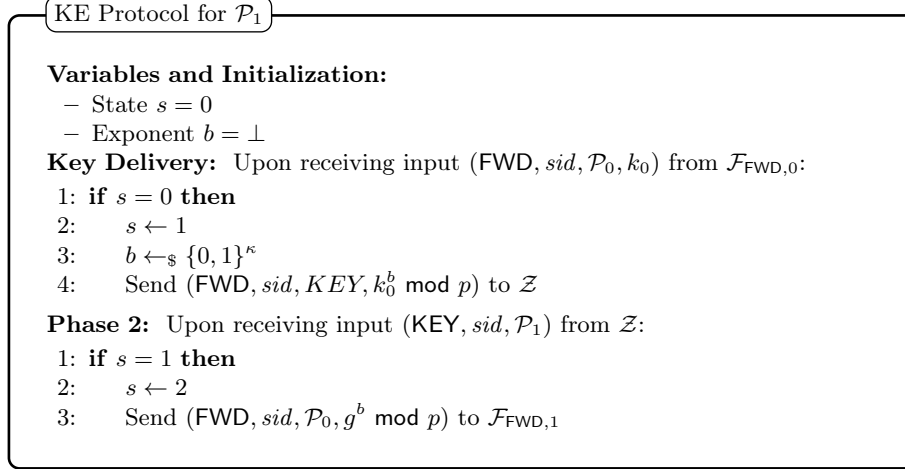


Fig. 6: The counterpart of the Diffie-Hellman key exchange protocol in the real world.

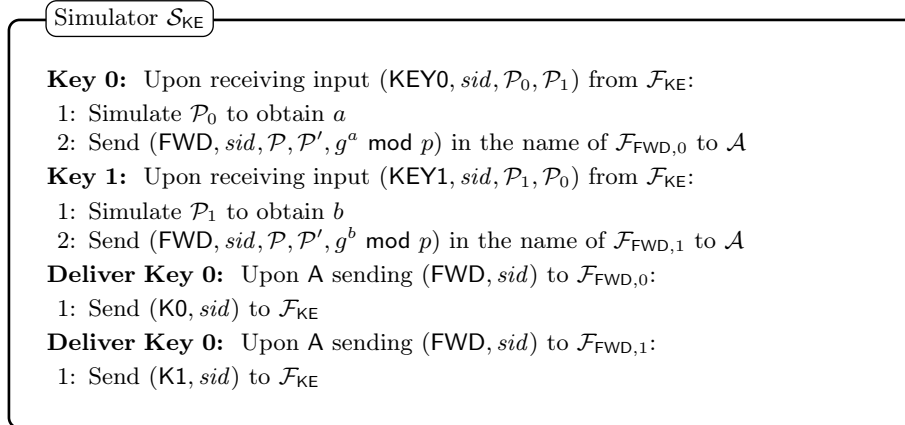


Fig. 7: The simulator $\mathcal{S}_{\text{KE}}$ for $\mathcal{F}_{\text{KE}}$ intercept messages from and to $\mathcal{A}$ in the real world to ensure the ideal world's transcript matches the real world's transcript.

key and a message as input and outputs a ciphertext, and a decryption algorithm $D : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$ taking a ciphertext and key as input and outputs a message.

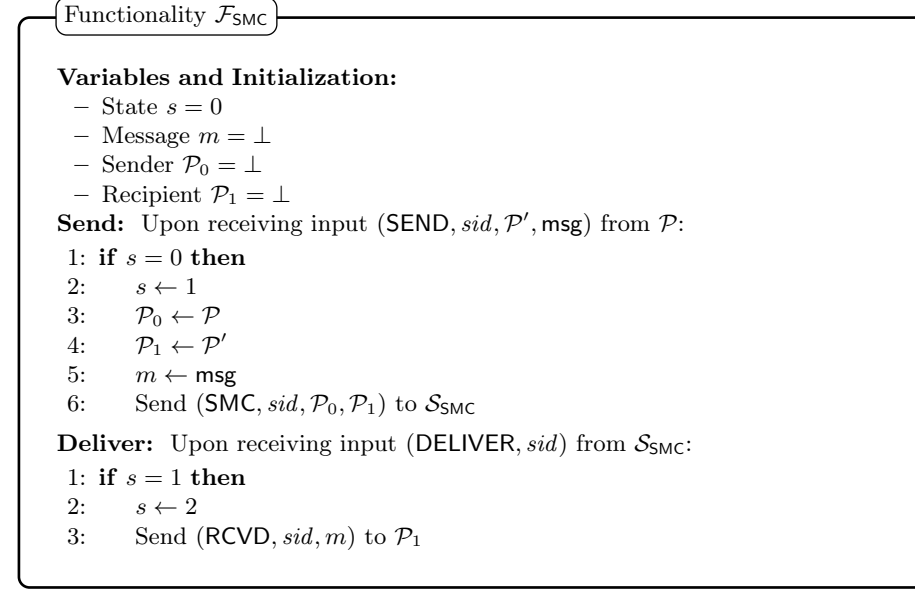


Fig. 8: The secure message communication Functionality.

SMC Protocol for $\mathcal{P}_0$

Variables and Initialization:

- State $s = 0$
- Message $m = \perp$

Send: Upon receiving input (SEND, $sid, \mathcal{P}', \text{msg}$) from $\mathcal{P}$:

- 1: **if** $s = 0$ **then**
- 2: $s \leftarrow 1$
- 3: $m \leftarrow \text{msg}$
- 4: Send (KEY, sid) to $\mathcal{F}_{KE}$

Receive Key: Upon receiving input (RCVD, sid, k) from $\mathcal{F}_{KE}$:

- 1: **if** $s = 1$ **then**
- 2: $s \leftarrow 2$
- 3: Send (FWD, $sid, E(k, m)$) to $\mathcal{F}_{FWD}$

Fig. 9: The secure message communication protocol executed by party $\mathcal{P}_1$.

SMC Protocol for $\mathcal{P}_1$

Variables and Initialization:

- State $s = 0$
- Key $k = \perp$

Key Exchange: Upon receiving input (RCVD, sid, k') from $\mathcal{F}_{KE}$:

- 1: **if** $s = 0$ **then**
- 2: $s \leftarrow 1$
- 3: $k \leftarrow k'$
- 4: Send (KEY, sid) to $\mathcal{F}_{KE}$

Receive: Upon receiving input (FWD, sid, c) from $\mathcal{F}_{FWD}$:

- 1: **if** $s = 1$ **then**
- 2: $s \leftarrow 2$
- 3: Send (RCVD, $sid, D(k, c)$) to $\mathcal{Z}$

Fig. 10: The secure message communication protocol executed by party $\mathcal{P}_2$.

Simulator $\mathcal{S}_{\text{SMC}}$

Variables and Initialization:

- State $s = 0$
- Key $k = \perp$
- Random numbers $c \leftarrow_{\S} \{0, 1\}^\kappa$

Start KE Phase 1: Upon receiving input $(\text{SMC}, \text{sid}, \mathcal{P}_0, \mathcal{P}_1)$ from $\mathcal{F}_{\text{SMC}}$:

- 1: **if** $s = 0$ **then**
- 2: $s \leftarrow 1$
- 3: Send $(\text{KEY0}, \text{sid})$ to $\mathcal{A}$ in the name of $\mathcal{F}_{\text{KE}}$

Start KE Phase 2: Upon $\mathcal{A}$ sending $(\text{KEY0}, \text{sid})$ to $\mathcal{F}_{\text{KE}}$:

- 1: **if** $s = 1$ **then**
- 2: $s \leftarrow 2$
- 3: Send $(\text{KEY1}, \text{sid})$ to $\mathcal{A}$ in the name of $\mathcal{F}_{\text{KE}}$

Forward cipher: Upon $\mathcal{A}$ sending $(\text{KEY1}, \text{sid})$ to $\mathcal{F}_{\text{KE}}$:

- 1: **if** $s = 2$ **then**
- 2: $s \leftarrow 3$
- 3: Send $(\text{FWD}, \text{sid}, g^r \bmod p, \mathcal{P}_0, \mathcal{P}_1)$ to $\mathcal{A}$ in the name of $\mathcal{F}_{\text{FWD}}$

Deliver cipher: Upon $\mathcal{A}$ sending $(\text{KEY1}, \text{sid})$ to $\mathcal{F}_{\text{KE}}$:

- 1: **if** $s = 3$ **then**
- 2: $s \leftarrow 4$
- 3: Send $(\text{DELIVER}, \text{sid})$ to $\mathcal{F}_{\text{SMC}}$

Fig. 11: The simulator for $\mathcal{F}_{\text{SMC}}$.